

MarketHub Backend Project – Step-by-Step Guide

This PDF contains the complete backend setup and API development flow for your Java + Spring Boot + MySQL marketplace project.

1. Project Overview

Project Name: MarketHub

Features:

- User Signup/Login
- Add Product (Sell / Rent)
- View Products
- Search Product by ID
- Update Product
- Delete Product
- MySQL Database Integration

2. Required Tools

Install and verify:

- Java JDK 17
- Maven
- MySQL + MySQL Workbench
- VS Code
- Java Extension Pack
- Spring Boot Extension Pack

3. Spring Boot Project Creation

Use Spring Initializr (<https://start.spring.io/>) with:

- Project: Maven
- Language: Java
- Spring Boot: 3.3.5
- Group: com.markethub
- Artifact: backend
- Packaging: Jar
- Java: 17

Dependencies:

- Spring Web
- Spring Data JPA
- MySQL Driver
- Lombok

4. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
    <relativePath/>
  </parent>

  <groupId>com.markethub</groupId>
  <artifactId>backend</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>backend</name>
  <description>MarketHub Backend</description>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <scope>runtime</scope>
    </dependency>

    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>

```

5. Database Setup

```
CREATE DATABASE markethub;
```

6. application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/markethub
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

server.port=8080
```

7. HomeController Test API

```
package com.markethub.backend.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "MarketHub Backend Running ■";
    }
}
```

8. User Registration + Login APIs

Create the following files under the correct packages:

```
model/User.java
model/LoginRequest.java
repository/UserRepository.java
service/UserService.java
controller/UserController.java
```

User.java

```
package com.markethub.backend.model;

import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private String password;

    public User() {}

    public User(String name, String email, String password) {
        this.name = name;
        this.email = email;
        this.password = password;
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

LoginRequest.java

```
package com.markethub.backend.model;

public class LoginRequest {
    private String email;
    private String password;

    public LoginRequest() {}

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

UserRepository.java

```
package com.markethub.backend.repository;
```

```
import com.markethub.backend.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}
```

UserService.java

```
package com.markethub.backend.service;

import com.markethub.backend.model.User;
import com.markethub.backend.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.Optional;

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public User registerUser(User user) {
        return userRepository.save(user);
    }

    public String loginUser(String email, String password) {
        Optional<User> userOptional = userRepository.findByEmail(email);

        if (userOptional.isPresent()) {
            User user = userOptional.get();

            if (user.getPassword().equals(password)) {
                return "Login Successful";
            } else {
                return "Invalid Password";
            }
        } else {
            return "User Not Found";
        }
    }
}
```

UserController.java

```
package com.markethub.backend.controller;

import com.markethub.backend.model.LoginRequest;
import com.markethub.backend.model.User;
import com.markethub.backend.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/users")
@CrossOrigin("")
public class UserController {

    @Autowired
    private UserService userService;

    @PostMapping("/register")
    public User registerUser(@RequestBody User user) {
        return userService.registerUser(user);
    }
}
```

```
@PostMapping("/login")
public String loginUser(@RequestBody LoginRequest loginRequest) {
    return userService.loginUser(loginRequest.getEmail(), loginRequest.getPassword());
}
}
```

9. Product APIs

model/Product.java
repository/ProductRepository.java
service/ProductService.java
controller/ProductController.java

Product.java

```
package com.markethub.backend.model;

import jakarta.persistence.*;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String category;
    private String type; // SELL or RENT
    private double price;
    private String description;

    public Product() {}

    public Product(String name, String category, String type, double price, String description) {
        this.name = name;
        this.category = category;
        this.type = type;
        this.price = price;
        this.description = description;
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }
    public String getType() { return type; }
    public void setType(String type) { this.type = type; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
}
```

ProductRepository.java

```
package com.markethub.backend.repository;

import com.markethub.backend.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository extends JpaRepository<Product, Long> {
}
```

ProductService.java

```
package com.markethub.backend.service;
import com.markethub.backend.model.Product;
```

```

import com.markethub.backend.repository.ProductRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class ProductService {

    @Autowired
    private ProductRepository productRepository;

    public Product addProduct(Product product) {
        return productRepository.save(product);
    }

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    public Product getProductById(Long id) {
        Optional<Product> product = productRepository.findById(id);
        return product.orElse(null);
    }

    public String deleteProduct(Long id) {
        if (productRepository.existsById(id)) {
            productRepository.deleteById(id);
            return "Product deleted successfully";
        } else {
            return "Product not found";
        }
    }

    public Product updateProduct(Long id, Product updatedProduct) {
        Optional<Product> existingProductOpt = productRepository.findById(id);

        if (existingProductOpt.isPresent()) {
            Product existingProduct = existingProductOpt.get();

            existingProduct.setName(updatedProduct.getName());
            existingProduct.setCategory(updatedProduct.getCategory());
            existingProduct.setType(updatedProduct.getType());
            existingProduct.setPrice(updatedProduct.getPrice());
            existingProduct.setDescription(updatedProduct.getDescription());

            return productRepository.save(existingProduct);
        } else {
            return null;
        }
    }
}

```

ProductController.java

```

package com.markethub.backend.controller;

import com.markethub.backend.model.Product;
import com.markethub.backend.service.ProductService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
@CrossOrigin("")
public class ProductController {

    @Autowired

```



```
private ProductService productService;

@PostMapping("/add")
public Product addProduct(@RequestBody Product product) {
    return productService.addProduct(product);
}

@GetMapping("/all")
public List<Product> getAllProducts() {
    return productService.getAllProducts();
}

@GetMapping("/{id}")
public Product getProductById(@PathVariable Long id) {
    return productService.getProductById(id);
}

@DeleteMapping("/delete/{id}")
public String deleteProduct(@PathVariable Long id) {
    return productService.deleteProduct(id);
}

@PutMapping("/update/{id}")
public Product updateProduct(@PathVariable Long id, @RequestBody Product product) {
    return productService.updateProduct(id, product);
}
}
```

10. API Testing Examples

Register User

POST http://localhost:8080/api/users/register

```
{
  "name": "Deepak",
  "email": "deepak@gmail.com",
  "password": "123456"
}
```

Login User

POST http://localhost:8080/api/users/login

```
{
  "email": "deepak@gmail.com",
  "password": "123456"
}
```

Add Product

POST http://localhost:8080/api/products/add

```
{
  "name": "Yamaha Bike",
  "category": "Vehicle",
  "type": "RENT",
  "price": 5000,
  "description": "Bike available for rent in good condition"
}
```

Get All Products

GET http://localhost:8080/api/products/all

Get Product by ID

GET http://localhost:8080/api/products/1

Update Product

PUT http://localhost:8080/api/products/update/1

```
{
  "name": "Updated Bike",
  "category": "Vehicle",
  "type": "SELL",
  "price": 60000,
  "description": "Updated description - good condition"
}
```

Delete Product

DELETE http://localhost:8080/api/products/delete/1